

Project Report

Student Management System

A Python-Based Application for Managing Student Records

Internship : Python Development Internship (Unpaid, 1 Month)

Project Title	Student Management System
Technology Used	Python 3 (Standard Library)
Submitted By	[Thanishaa B]
Submission Date	August 2026

1. Introduction

Educational institutions handle large volumes of student data on a daily basis, including personal details, enrolled courses, and academic performance. Managing this information manually using registers or spreadsheets is time-consuming and prone to errors. The Student Management System is a command-line Python application designed to simplify this process by allowing administrative staff to add, view, search, update, and delete student records efficiently through a simple, menu-driven interface.

This project was developed as part of a one-month Python Development Internship, with the goal of applying core Python concepts - Object-Oriented Programming, file handling, and exception handling - to a practical, real-world problem.

2. Objectives

- Design and implement a fully functional student record management application.
- Apply Object-Oriented Programming principles such as encapsulation and abstraction.
- Persist data reliably to disk using file handling (JSON), so records are not lost between sessions.
- Implement robust exception handling for all foreseeable error conditions.
- Provide search and filter functionality for quick information retrieval.
- Follow a clean, modular project structure suitable for future extension (e.g. database or API integration).

3. Tools & Technologies Used

Category	Technology
Programming Language	Python 3.8+
Data Storage	JSON file (data/students.json)
Paradigm	Object-Oriented Programming (OOP)

Testing	unittest (Python standard library)
Version Control	Git & GitHub
IDE (suggested)	VS Code / PyCharm

4. System Architecture & Project Structure

The application follows a modular, layered architecture that separates data, business logic, utilities, and the user interface. This separation of concerns makes the codebase easier to read, test, and extend.

Layer / File	Responsibility
main.py	Menu-driven command-line interface (user interaction layer)
models/student.py	Student class - holds data and validation rules for a single record

services/student_manager.py	StudentManager class - CRUD operations, search, filter, statistics
utils/file_handler.py	JSONFileHandler class - generic JSON read/write persistence layer
utils/exceptions.py	Custom exception hierarchy for precise error handling
data/students.json	Auto-generated data file where all records are persisted
tests/test_student_manager.py	Unit tests covering CRUD, search, filter and persistence

5. Key Features

Add Student: Create a new student record with ID, name, age, course, email and grade, with full validation.

View All Students: Display all records in a clean, formatted table.

Search by ID: Instantly retrieve a specific student's details.

Search by Name: Case-insensitive, partial-match search by student name.

Update Student: Modify one or more fields of an existing record without affecting the rest.

Delete Student: Remove a record after explicit confirmation.

Filter by Course / Grade / Age Range: Narrow down the record list using common academic criteria.

Statistics: View total student count, average age, and grade-wise distribution.

Persistent Storage: All data is automatically saved to a JSON file after every change.

6. Object-Oriented Programming Concepts Applied

Concept	Application in this Project
Encapsulation	The Student class keeps its data and validation logic together, exposing a clean interface.
Abstraction	
Inheritance	All custom exceptions inherit from a common StudentManagementError base class.
Composition	StudentManager 'has a' JSONFileHandler instance rather than inheriting from it.
Class methods	

7. Exception Handling Strategy

Rather than relying on generic exceptions, the project defines a dedicated exception hierarchy in `utils/exceptions.py`, including `StudentNotFoundError`, `DuplicateStudentError`, `InvalidStudentDataError` and `FileOperationError`. Each of these is raised at the precise point where an error condition is detected (for example, attempting to fetch a non-existent student ID), and is caught in `main.py` to display a clear, user-friendly message - ensuring the application never crashes on invalid input.

8. Testing

The project includes a suite of automated unit tests (`tests/test_student_manager.py`) built with Python's `unittest` framework. The tests cover record creation, duplicate handling, validation errors, updates, deletion, search, filtering, and data persistence across separate program runs. All 9 test cases pass successfully, confirming the reliability of the core business logic.

Sample test run output:

```
$ python -m unittest discover tests
```

```
.....
```

```
-----
```

```
Ran 9 tests in 0.014s
```

```
OK
```

9. Sample Program Output

```
$ python main.py
=====
                        STUDENT MANAGEMENT SYSTEM
=====
1. Add Student          2. View All Students
3. Search Student by ID  4. Search Students by Name
5. Update Student        6. Delete Student
7. Filter by Course      8. Filter by Grade
9. Filter by Age Range   10. View Statistics
0. Exit
=====
Enter your choice: 1

--- Add New Student ---
Student ID: S101
Name: Thanishaa B
Age: 21
Course: Computer Science
Email (optional): thani@example.com
Grade (A/B/C/D/F, optional): A

Student added successfully:  [S101] Thanishaa | Age: 21 | Course: Computer
Science | Grade: A | Email:
thani@example.com
```

10. Challenges Faced & Solutions

Ensuring data integrity after every operation: Solved by writing the entire in-memory dataset back to the JSON file immediately after any Create/Update/Delete operation.

Preventing duplicate or invalid records: Solved by validating all input inside the Student class constructor before it is ever stored.

Keeping the interface crash-proof: Solved with a comprehensive try/except structure in main.py mapped to a custom exception hierarchy.

11. Conclusion

The Student Management System successfully demonstrates the practical application of core Python concepts - OOP design, file-based persistence, and structured exception handling - within a clean, modular codebase. The application fulfills all required key features (CRUD operations, search, filtering, and robust error handling) and is organized in a way that makes it straightforward to extend with a database backend, a REST API, or a graphical interface in the future. Completing this project strengthened practical skills in software design, testing, and writing production-quality, maintainable Python code.

12. Future Enhancements

- Migrate storage from JSON to a relational database (SQLite/MySQL) for scalability.
- Expose the same StudentManager logic through a REST API using Flask or FastAPI.

- Add CSV/Excel import and export.
- Build a simple GUI using Tkinter or a web front-end.
- Add user authentication for multi-user / admin access control.

14. Repository Link

GitHub Repository: <https://github.com/thanishaab-21/student-management-system.git>